

ATLAS

Personal Life Operating System

Architecture Document -- v0.1.0

Repository: <https://github.com/Swayamjain1240/Atlas>

Confidential & Proprietary

Table of Contents

- 1. Executive Summary
- 2. High-Level Architecture Overview
- 3. System Components
- 4. Data Flow Architecture
- 5. Deployment Architecture
- 6. Security Architecture
- 7. Technology Stack
- 8. Scalability Considerations

1. Executive Summary

Atlas is a local-first, AI-powered Personal Life Operating System designed to solve the fundamental problem of digital knowledge fragmentation. Modern knowledge workers operate across 13+ applications daily, losing approximately 1 hour per day searching for their own information. Atlas creates a unified memory layer that indexes files, browser history, messages, emails, calendar events, and code repositories -- then enables natural language querying over all of it.

The architecture follows a modular monolithic pattern with clear separation of concerns: an ingestion pipeline, vector storage layer, RAG-based AI engine, REST API, and a React frontend with 3D visualizations. All data processing happens locally on the user's machine, ensuring complete privacy and data sovereignty.

2. High-Level Architecture Overview

Atlas uses a four-layer architecture pattern:

Layer 1: Ingestion Layer

Responsible for discovering, reading, and chunking data from various sources. Each source type (files, browser history, messages, email) has a dedicated ingestor that normalizes data into a common Document format.

Layer 2: Storage Layer

A hybrid storage system combining:

- ChromaDB vector database for semantic search over embeddings
- SQLite for metadata, configuration, and relationship tracking
- File system cache for raw content

Layer 3: AI Engine

The core intelligence layer using Retrieval-Augmented Generation (RAG):

- Sentence-Transformers for local embedding generation
- Ollama for local LLM inference (Llama 3.2, Mistral, etc.)
- ChromaDB vector store for similarity search
- Context assembly and prompt engineering pipeline

Layer 4: API & Presentation

FastAPI REST backend exposing endpoints for query, ingestion, and memory management. React frontend with Three.js 3D knowledge graph visualization, chat interface, timeline view, and analytics dashboard.

3. System Components

AtlasAIEngine

Core RAG pipeline. Embeds documents, retrieves context, generates answers via local LLM. Handles vector store operations.

FileIngestor

Watches configured directories, reads supported file types (.txt, .md, .pdf, .docx, code files), chunks content, converts to Documents.

BrowserHistoryIngestor

Reads Chrome-based browser history databases (Chrome, Edge, Brave). Extracts URLs, titles, visit counts, timestamps.

MessageIngestor

Planned: ingests Slack, Discord, WhatsApp exports, and email via IMAP. Extracts conversations, participants, timestamps.

FastAPI Server

REST API with endpoints: /api/v1/query, /api/v1/ingest, /api/v1/stats, /api/v1/search, /api/v1/health.

React Frontend

Single-page application with Three.js 3D knowledge graph, chat interface, timeline, dashboard, and settings panels.

Security Module

API key validation, CORS enforcement, input sanitization, rate limiting, security headers, auth middleware.

4. Data Flow Architecture

The primary data flow for a user query:

- 1. User enters query in chat interface
- 2. Frontend sends POST /api/v1/query with natural language question
- 3. FastAPI validates input (sanitization, length limits, rate limiting)
- 4. AtlasAIEngine.retrieve() embeds query using Sentence-Transformers
- 5. ChromaDB vector store performs cosine similarity search
- 6. Top-N most relevant documents are returned with metadata
- 7. AtlasAIEngine.generate() assembles context + query into prompt
- 8. Ollama local LLM generates natural language answer
- 9. Response returned with answer, sources, and processing time
- 10. Frontend renders answer in chat with source citations

5. Deployment Architecture

Atlas is designed for local-first deployment on the user's machine. No cloud dependency. No data leaves the device.

Minimum Requirements

- CPU: 4+ cores (x86_64 or ARM)
- RAM: 8GB minimum, 16GB recommended
- Storage: 10GB free for data store (scales with usage)
- GPU: Optional -- for faster local LLM inference
- OS: Windows 10+, macOS 12+, Linux (Ubuntu 20.04+)

Containerized Deployment (Advanced)

Docker Compose with services: atlas-api, atlas-ollama, atlas-frontend. GPU passthrough available for accelerated inference.

6. Security Architecture

Security is embedded throughout the architecture, not bolted on at the end:

- All processing is LOCAL -- no data sent to external servers
- API keys stored in environment variables, never in code
- Input validation and sanitization on all API endpoints
- Strict CORS allow list (localhost only)
- Rate limiting to prevent abuse
- Security headers (X-Content-Type-Options, X-Frame-Options, CSP)
- No debug mode in production
- Regular dependency vulnerability scanning
- Git history scanned for leaked secrets pre-commit
- Authentication via local API key or biometric (planned)

7. Technology Stack

Backend

Python 3.11+, FastAPI, Uvicorn

AI/ML

Sentence-Transformers, ChromaDB, Ollama, LlamaIndex

Embedding Model

all-MiniLM-L6-v2 (384-dim, local)

Local LLM

Llama 3.2 3B via Ollama

Database

ChromaDB (vectors), SQLite (metadata)

Frontend

React 18, Vite, Tailwind CSS, Three.js, Recharts

Desktop

Tauri (Rust) -- planned for v1.1

Auth

Argon2id (planned), API key-based for v1.0

Container

Docker, Docker Compose

8. Scalability Considerations

Atlas is designed to scale from personal use to team deployments:

- Vector store supports millions of embeddings with HNSW indexing
- Async ingestion pipeline for concurrent document processing
- Caching layer for frequent queries
- Planned: distributed mode for team knowledge bases
- Planned: plugin architecture for custom ingestors and agents
- Planned: MCP (Model Context Protocol) support for tool integration